NHK Recipe State Probing: A Clean Benchmark for Tracking Ingredient States in Recipes

Anonymous Authors

Abstract

Large language models are increasingly evaluated on procedural text, but most existing benchmarks emphasize surface structure such as step ordering, retrieval, or alignment rather than the latent world states that procedures describe. We introduce NHK Recipe State Probing, a benchmark for ingredient-centric state tracking built from approximately 25K professionally edited Japanese recipes from NHK's "Minna no Kyo no Ryori." The benchmark combines one large automatically derived task, Step Reference Resolution, with two manually annotated state-probing tasks on a 100-recipe subset: Ingredient Usage Prediction and Ingredient Tracing. The annotation represents each recipe as a stepwise world state together with explicit transformation links between pre- and post-step ingredient items, yielding an ingredient state graph. Across text-only and vision-language models from 7B to 72B parameters, the benchmark reveals a consistent gap between recognizing local textual cues and maintaining correct ingredient state, especially under subtraction, splitting, and parallel preparation. Fine-tuning on the small annotated split helps strong models substantially, but often hurts smaller models, indicating that the benchmark remains challenging even with direct supervision. These results position NHK Recipe State Probing as a targeted and reproducible test bed for studying grounded procedural understanding in recipes.

1 Introduction

Understanding procedural text requires more than identifying which step comes first or which sentence refers to another. A procedure also implies a latent world state that changes over time: ingredients are cut, combined, divided, heated, discarded, or reserved for later use. This type of state tracking is central to human understanding of instructions and event scripts [10, 19], yet it remains difficult to evaluate directly in contemporary language models.

Prior work on recipes and procedures has produced valuable datasets for cross-modal retrieval [7, 9], multimodal comprehension [16], flow-graph induction [8, 17], state change extraction [11], and state probing [12]. However, these resources leave an evaluation gap. Large recipe corpora are often noisy or weakly structured, while state-tracking datasets are typically open-domain, small in a specific domain, or not tied to professionally authored instructions and images. In addition, many benchmarks focus on event ordering or goal-step relations [13, 14, 20] rather than whether a model can maintain a faithful representation of intermediate ingredient states.

This paper introduces *NHK Recipe State Probing*, a benchmark designed specifically for that missing measurement problem. We start from approximately 25K recipes crawled from NHK Educational's "Minna no Kyo no Ryori," a professionally curated Japanese recipe source with standardized ingredient lists, explicit HTML step references, and final-dish images. We then annotate a 100-recipe subset with stepwise world states and fine-grained transformation links, producing an *ingredient state graph* for each recipe. The resulting benchmark contains three tasks with complementary difficulty profiles: Step Reference Resolution, Ingredient Usage Prediction, and Ingredient Tracing.

The benchmark is motivated by a simple observation illustrated in Figure 1: recipe text exposes only a partial view of the cooking process, while humans implicitly reconstruct the underlying world state. Our benchmark makes that latent structure explicit and uses it to evaluate whether models can do the same. The key contribution is therefore not a new model, but a clean, state-centric evaluation framework.

We make four contributions. First, we construct a high-quality recipe benchmark grounded in professionally edited NHK recipes, rather than loosely curated web text. Second, we define an annotation protocol that records stepwise world states and transformation links, enabling exact state-probing tasks beyond ordering or retrieval. Third, we release three benchmark tasks that vary in supervision cost and difficulty: one large-scale automatically derived task and two manually validated probing tasks. Fourth, we provide a broad empirical study over LLMs and LMMs showing that current systems, including strong closed and open models, still fail on important aspects of ingredient state tracking.

2 Related Work

Recipe datasets and cooking representations. Recipe understanding has been studied through large multimodal corpora such as Recipe1M and Recipe1M+ [7, 9], multimodal question answering in RecipeQA [16], and structured recipe representations such as the English Recipe Flow Graph Corpus and MM-ReS [8, 17]. These resources are important, but they primarily target retrieval, comprehension, or workflow construction. NHK Recipe State Probing instead centers evaluation on stepwise ingredient state, using explicit human annotations of intermediate items and their transformations.

Procedural state tracking and condition inference. OPENPI [11] and action condition inference [15] directly address state change or pre/postcondition modeling in procedural text, while Baked-in State Probing [12] studies whether language models can expose latent world-state knowledge through prompting. Our benchmark is complementary to these efforts: it focuses on recipes, uses professionally authored instructional text, and provides state annotations tailored to ingredient persistence, combination, splitting, and usage. Compared with generic open-domain procedural resources, it offers tighter control over entity normalization and visual grounding.

Procedural ordering, graph induction, and multimodal reasoning. Recent work also evaluates temporal ordering, goal-step reasoning, hierarchical procedures, multimodal sequencing, and procedural graph extraction [2, 14, 20–22]. These benchmarks reveal weaknesses in event sequencing and structural prediction, but they do

AI-Generated

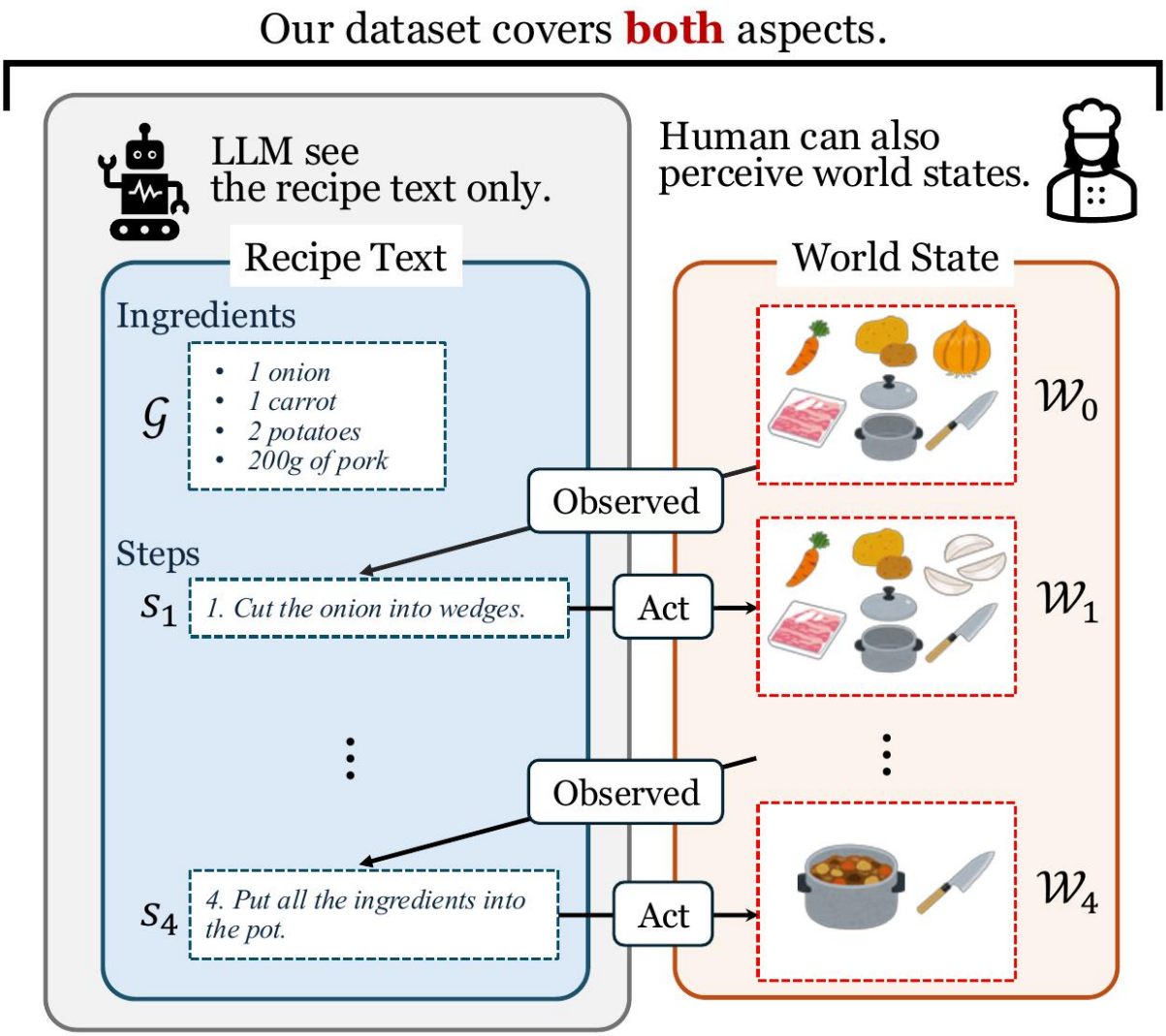


Figure 1: Conceptual overview of NHK Recipe State Probing. Recipes expose ingredients and step text, while the underlying world states must be inferred. Our benchmark makes those state transitions explicit and evaluates whether language-only and multimodal models can recover them.

not directly ask whether a model knows which intermediate object still contains a target ingredient, or whether an ingredient has already been consumed by a given step. Our tasks fill that gap and expose failures that are not visible in ordering-centric evaluations.

3 Benchmark Design

3.1 Source collection

Our source corpus is drawn from "Minna no Kyo no Ryori" (https://www.kyounoryouri.jp/), a Japanese recipe site operated by NHK Educational. The crawl contains approximately 25K recipes. The source is especially suitable for benchmark construction for three reasons. First, the recipes are professionally edited rather than community-authored, which reduces noise in ingredient naming and step formatting. Second, ingredient lists are standardized and quantities are explicitly stated. Third, the HTML contains step-reference markup that enables automatic construction of reference-resolution instances. Final-dish photographs are also available, making it possible to define a multimodal variant without requiring per-step image alignment.

3.2 Ingredient state graphs

For each annotated recipe, we define an initial world state $\mathcal{W}_0$ as the set of ingredients in the recipe list. Each step s_t transforms the current state into a new state:

$$\mathcal{W}_{t+1} = (\mathcal{W}_t \setminus \mathcal{U}_t) \cup \mathcal{V}_t, \tag{1}$$

Table 1: Benchmark summary. The annotated subset contains 100 recipes split into 60 training recipes and 40 evaluation recipes.

Resource / Task	Size	Chance
Crawled NHK recipes	~25K	–
Annotated recipes	100	–
Step Reference Resolution	33,106	0.348 avg.
Ingredient Usage Prediction	346	0.500
Ingredient Tracing	333	≤0.193 avg.

where $\mathcal{U}_t$ is the set of items modified by step t and $\mathcal{V}_t$ is the set of resulting items after the step. The annotation additionally records fine-grained input-output links (u, v) between elements of $\mathcal{U}_t$ and $\mathcal{V}_t$. This supports one-to-one changes such as "egg" → "beaten egg," many-to-one merges such as "milk" + "sugar" → "milk and sugar mixture," and one-to-many splits such as reserving part of a marinade. Chaining these links across time yields an ingredient state graph.

3.3 Tasks

Figure 2 illustrates the underlying state representation; the benchmark exposes it through three tasks.

Step Reference Resolution. Many NHK recipes explicitly refer back to earlier steps. We mask such a reference in a target step and ask the model to predict the referenced step index. This task is automatically derived from the large crawl and does not require manual state annotation. It probes whether models can recover local procedural dependencies from the surrounding context.

Ingredient Usage Prediction. Given a recipe and a target raw ingredient g at step τ, the model predicts whether g remains unused by the end of that step. The task is binary, with a chance rate of 0.5. Correct prediction requires distinguishing between ingredients that have not yet participated in any transformation and ingredients that have already been absorbed into an intermediate item.

Ingredient Tracing. Given a target raw ingredient g and the world-state items present at step τ, the model must select which item or items contain g. This is the most direct state-probing task because the correct answer can involve persistence through transformation, merging, or splitting. The random baseline depends on the number of candidate items and is upper-bounded by $1/|\mathcal{W}_\tau|$, averaging approximately 0.193 in our data.

All tasks use the recipe title, ingredient list, and step list as textual input. The multimodal setting prepends the final dish image to the prompt. Following the benchmark design goal of minimizing prompt engineering confounds, we require direct answer-only outputs and do not use chain-of-thought prompting or in-context demonstrations.

4 Annotation Protocol and Benchmark Validation

4.1 Sampling and annotation process

We manually annotate 100 recipes sampled from the larger crawl. The sample is selected to flatten the otherwise biased distribution of step-reference distances, improving diversity in local and long-range dependencies. Two human annotators create the annotations. For each recipe, annotators read the title T, ingredients G, and instructions S, then iteratively write the post-step world state after each instruction and record the transformation links between consumed and produced items.

The annotations are created in a spreadsheet-based workflow with automated consistency checks. These checks prevent unintended item creation or disappearance, verify that transformed items are accounted for at every step, and make it easier to inspect the full input-output chain of each ingredient. The public release contains 100 annotated recipes with a 60/40 train/evaluation split: https://huggingface.co/datasets/mashi6n/nhkrecipe-100-anno-1.

4.2 What the benchmark measures

The benchmark is intended to measure a model's ability to maintain and query latent ingredient states in procedural text. The three tasks target complementary facets of this capability. Step Reference Resolution captures local dependency recovery from explicit references. Ingredient Usage Prediction captures whether a raw ingredient has been incorporated into the evolving world state. Ingredient Tracing captures persistence under transformations and is the closest proxy to full ingredient state tracking.

Several design choices improve measurement quality. The source recipes are professionally edited, reducing noise relative to open web corpora [5]. Ingredient names are standardized, which lowers ambiguity in the initial state. The manually annotated tasks are based on explicit state transitions rather than weak heuristics. Finally, the benchmark exposes both textual and image-conditioned variants without requiring separate datasets.

The benchmark is also reproducible in a stronger sense than many procedural datasets built from noisy web extraction. Because the source pages contain stable ingredient lists and explicit step boundaries, the initial state and the sequence of update decisions are inspectable from the raw recipe alone. This makes it possible to audit benchmark instances back to the originating recipe and to distinguish source ambiguity from annotation error when a model prediction appears disputable.

The benchmark is also deliberately diagnostic rather than monolithic. Step Reference Resolution can be solved from textual context alone, so it tests whether the benchmark captures ordinary procedural dependency recovery at scale. Ingredient Usage Prediction adds a minimal latent-state question with a binary answer, making it useful as a controlled probe for whether a model can reason about ingredient consumption rather than just lexical mention. Ingredient Tracing is the strongest test because it requires carrying ingredient identity through merges, splits, and renamings. A model that performs well on all three tasks is therefore not merely good at local matching; it is better at maintaining a coherent latent representation of recipe state.

AI-Generated

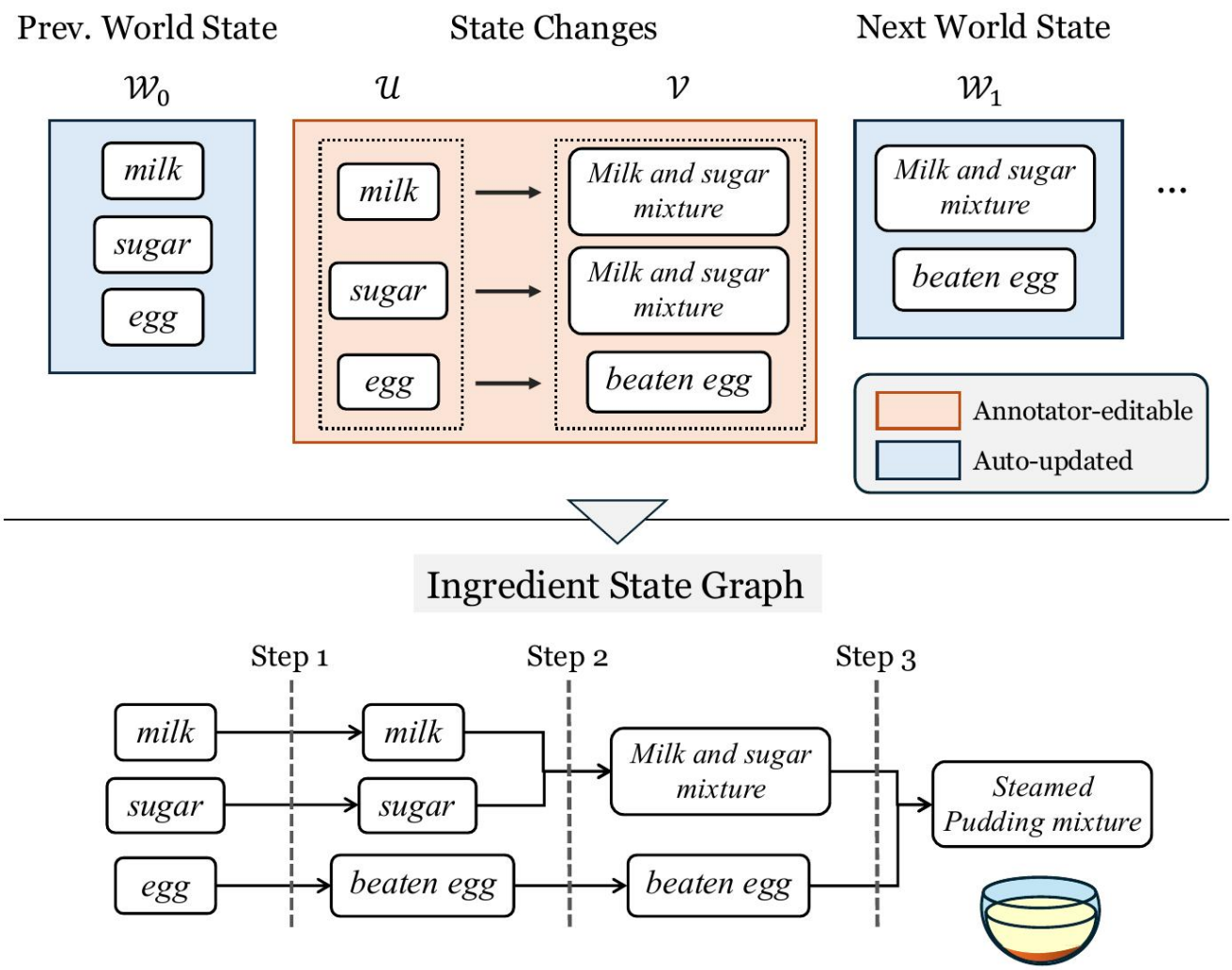


Figure 2: Annotation interface and resulting ingredient state graph. Annotators edit the modified items and their outputs at each step, while the spreadsheet automatically updates the world state and checks for consistency.

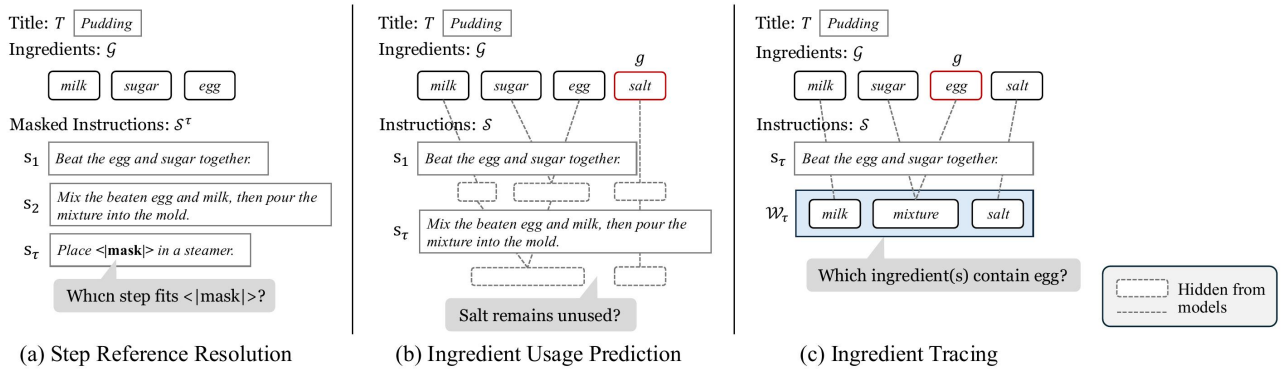
This task decomposition also makes the benchmark easier to interpret than a single end-to-end score. If a model succeeds on Step Reference Resolution but fails on Ingredient Usage Prediction, the likely weakness is not basic procedural ordering but bookkeeping over whether an entity has already been consumed. If it succeeds on usage prediction but fails on Ingredient Tracing, the remaining gap is finer-grained identity tracking through renaming, merging, or branching. The suite therefore supports error analysis at the level of representational failure mode rather than only aggregate accuracy.

We also inspected whether the annotated tasks could be reduced to surface-form overlap. Many tracing instances cannot: the correct carrier item is often a mixture, sauce, batter, or reserved portion whose name no longer repeats the raw ingredient string. These cases are where performance drops most sharply, which is useful evidence that the benchmark is not dominated by lexical matching and that failure reflects difficulty with intermediate state reconstruction.

4.3 Known limits of the annotation

The benchmark also has limitations that matter for interpretation. The manually annotated subset is small, which is appropriate for careful state annotation but constrains statistical power on fine-grained slices. We do not report inter-annotator agreement, so reliability is supported by the consistency-checking workflow and manual verification rather than a formal agreement statistic. In addition, the source comes from a single professional Japanese recipe domain; this improves cleanliness but narrows topical coverage and leaves open questions about transfer to noisier or more diverse procedural text.

Another important nuance is that only the annotated tasks directly expose latent ingredient state. Step Reference Resolution is



(a) Step Reference Resolution (b) Ingredient Usage Prediction (c) Ingredient Tracing

Figure 3: The three benchmark tasks. Step Reference Resolution masks an explicit backward reference in a step and asks for the referenced step index. Ingredient Usage Prediction asks whether a target ingredient has remained unused by step τ. Ingredient Tracing asks which intermediate item or items at step τ still contain a target ingredient. Dotted boxes mark information hidden from the model.

AI-Generated

automatically derived and closer to classic procedural dependency prediction. This is intentional: the benchmark is meant to cover both a broad large-scale task and smaller, more expensive human-validated state-probing tasks. We therefore recommend using the suite jointly rather than drawing conclusions from any single task in isolation.

4.4 Why the task suite is diagnostic

The three tasks are intentionally ordered by how much latent state must be recovered from the same recipe context. Step Reference Resolution can often be answered from local textual evidence and therefore validates that the benchmark still covers ordinary procedural dependency recovery. Ingredient Usage Prediction adds temporal bookkeeping over a specific raw ingredient. Ingredient Tracing is harder because the answer space is set-valued and the target ingredient may survive only through renamed, merged, or split intermediate items. This progression gives the benchmark diagnostic value: success on the first task alone indicates sensitivity to explicit references, while success on all three tasks is stronger evidence of persistent ingredient-state tracking.

The underlying annotations also make it difficult to solve the benchmark with simple lexical heuristics. After a step, a target ingredient may persist only through an intermediate carrier item such as a batter, sauce, or reserved portion whose surface form no longer matches the raw ingredient name. Conversely, an ingredient can disappear from the active state after disposal even if it was mentioned prominently in the preceding text. The evaluation therefore distinguishes sentence-level mention matching from genuine state maintenance over entity identity, composition, and temporal scope. Because the released supervision is an explicit state graph rather than only final labels, model errors can be audited at the level of individual transitions.

5 Experimental Setup

We evaluate representative open and closed models, including Llama 3.1/3.3 [4], Swallow [3, 6], Qwen2.5 [18], Qwen2.5-VL [1], and GPT-4o. The evaluated parameter scales range from 7B to 72B. We report both zero-shot prompting and fine-tuned results when available. Fine-tuning uses the benchmark's released training split and the same three task definitions as evaluation; the intent is not to maximize absolute performance through heavy optimization, but to test whether modest task-specific supervision improves state tracking.

The primary metric for all tasks is accuracy. For Ingredient Tracing, a prediction is correct only when the selected item set exactly matches the gold answer. Tables 2 and 3 mark statistically significant improvements from zero-shot to fine-tuning with a one-sided McNemar test at $p < 0.01$.

To keep the evaluation comparable across model families, each task uses a constrained answer format tied to the task definition. Step Reference Resolution expects a single step index, Ingredient Usage Prediction expects a binary decision, and Ingredient Tracing expects the full set of current-state items containing the target ingredient. We report taskwise scores alongside averages because the benchmark is intentionally non-redundant: the three tasks differ in answer space, supervision cost, and the amount of latent state recovery they require.

6 Benchmark Results

We first report text-only results, followed by multimodal results with the final dish image.

6.1 Overall trends

The benchmark separates models more clearly than chance on all three tasks, but the task profile matters. Step Reference Resolution is particularly challenging in zero shot for smaller models: Llama-3.1-8B achieves only 0.002 accuracy, far below the average chance rate of 0.348, while stronger models such as Qwen-2.5-72B and Swallow-3.1-70B reach 0.840 and 0.761, respectively. Ingredient Usage Prediction is easier for strong models but close to chance for several 7B systems. Ingredient Tracing yields high zero-shot scores for many large models, suggesting that some models can identify likely ingredient persistence when the candidate set is explicit, though the task is still far from saturated.

Table 2: Experimental results for LLMs. "Step Ref.", "Ingr. Usage", and "Ingr. Trace" denote Step Reference Resolution, Ingredient Usage Prediction, and Ingredient Tracing, respectively.

Model	Step Ref.	Ingr. Usage	Ingr. Trace	Ave.
Llama-3.1-8B	0.002	0.509	**0.462**	**0.324**
+ Fine Tuning	**0.184**†	**0.512**	0.111	0.269
Llama-3.1-70B	0.224	0.616	**0.835**	0.558
+ Fine Tuning	**0.708**†	**0.705**†	0.748	**0.720**
Llama-3.3-70B	0.595	0.520	0.844	0.653
+ Fine Tuning	**0.629**†	**0.566**†	**0.856**	**0.684**
Swallow-3.1-8B	0.330	**0.538**	**0.553**	**0.474**
+ Fine Tuning	**0.349**†	0.526	0.438	0.438
Swallow-3.1-70B	0.761	0.757	0.793	0.770
+ Fine Tuning	**0.809**†	**0.884**†	**0.817**	**0.837**
Swallow-3.3-70B	0.433	0.702	**0.880**	0.672
+ Fine Tuning	**0.495**†	**0.827**†	0.862	**0.728**
Qwen-2.5-7B	**0.199**	0.509	0.670	**0.459**
+ Fine Tuning	0.188	0.509	0.670	0.456
Qwen-2.5-14B	0.389	0.610	**0.763**	0.587
+ Fine Tuning	**0.568**†	**0.662**†	0.754	**0.661**
Qwen-2.5-32B	0.507	0.757	**0.817**	0.694
+ Fine Tuning	**0.659**†	**0.871**†	0.793	**0.774**
Qwen-2.5-72B	0.840	0.725	0.700	0.755
+ Fine Tuning	**0.871**†	**0.861**†	**0.712**	**0.815**
GPT-4o	0.776	0.812	0.883	0.824

† Statistically significant improvement over zero-shot baseline ($p < 0.01$, one-sided McNemar test).

Table 3: Experimental results for LMMs. All tasks and model training are conducted with images.

Model	Step Ref.	Ingr. Usage	Ingr. Trace	Ave.
Qwen-2.5-VL-7B	0.094	0.497	**0.523**	**0.371**
+ Fine Tuning	**0.151**†	**0.500**	0.234	0.295
Qwen-2.5-VL-32B	0.239	0.740	**0.745**	0.575
+ Fine Tuning	**0.280**†	**0.812**†	0.655	**0.582**
Qwen-2.5-VL-72B	0.883	0.662	0.886	0.810
+ Fine Tuning	**0.919**†	**0.945**†	**0.892**	**0.919**

† Statistically significant improvement over zero-shot baseline ($p < 0.01$, one-sided McNemar test).

Fine-tuning has a markedly scale-dependent effect. For large language models, it often produces substantial gains: Llama-3.1-70B improves from 0.558 average accuracy to 0.720, Qwen-2.5-32B from 0.694 to 0.774, and Swallow-3.1-70B from 0.770 to 0.837. In contrast, several smaller models degrade after fine-tuning, including Llama-3.1-8B (0.324 to 0.269) and Swallow-3.1-8B (0.474 to 0.438). This pattern suggests that limited direct supervision on state-probing tasks is useful only when the base model already has sufficient representational capacity.

The taskwise behavior is revealing. Fine-tuning consistently improves Step Reference Resolution for nearly every model family, often by large margins. By contrast, Ingredient Tracing sometimes decreases after fine-tuning even when the average improves. For example, Qwen-2.5-32B improves on Step Reference Resolution from 0.507 to 0.659 and on Ingredient Usage Prediction from 0.757 to 0.871, but its Ingredient Tracing score drops slightly from 0.817 to 0.793. The same trend appears in Llama-3.1-70B and Swallow-3.3-70B. This suggests that exact tracing remains the hardest component of the benchmark to optimize reliably from a small supervised split.

AI-Generated

6.2 Text-only versus multimodal models

The multimodal setting does not yield a uniform advantage. At 7B and 32B scale, Qwen-2.5-VL underperforms the corresponding text-only Qwen models on average in zero shot, and fine-tuning may still hurt Ingredient Tracing. At 72B, however, the multimodal model is the strongest system in the study: Qwen-2.5-VL-72B reaches 0.810 zero-shot average accuracy and 0.919 after fine-tuning, with especially strong scores on Step Reference Resolution (0.919) and Ingredient Usage Prediction (0.945). This indicates that the final-dish image can help, but only when the model is strong enough to integrate the visual cue with the procedural text.

This scale dependence is important for interpreting the role of images in recipe understanding. Because the benchmark provides only the final dish image, the visual signal is indirect: it cannot tell the model what happened at each intermediate step, but it can constrain the plausible end state. The strong performance of Qwen-2.5-VL-72B therefore suggests that high-capacity multimodal models can use the final visual outcome as a global consistency signal. The weaker results for smaller LMMs suggest that simply adding an image token does not automatically improve procedural state reasoning.

6.3 Comparison with a closed model

GPT-4o is a strong zero-shot baseline with 0.824 average accuracy, 0.776 on Step Reference Resolution, and 0.883 on Ingredient Tracing. Nevertheless, several open models are competitive or better on this benchmark. Swallow-3.1-70B with fine-tuning reaches 0.837 average accuracy, and Qwen-2.5-VL-72B with fine-tuning substantially exceeds GPT-4o on Step Reference Resolution while also improving the overall average to 0.919. The benchmark therefore remains discriminative even among high-performing contemporary systems.

7 Analysis and Discussion

7.1 What the benchmark reveals

Two findings stand out. First, explicit state supervision helps much more on dependency-heavy tasks than on already-strong tracing behavior. For example, Llama-3.1-70B improves dramatically on Step Reference Resolution from 0.224 to 0.708, while its Ingredient Tracing score decreases slightly from 0.835 to 0.748. This suggests that the annotation is particularly effective for teaching step-to-step dependency resolution and ingredient usage status, but exact tracing may require either more data or better inductive bias for set-valued state updates.

Second, strong overall scores should not be read as evidence that recipe state tracking is solved. Even the best models continue to fail on transformations that require subtraction, preservation of residue, or reasoning over parallel preparation streams. These are precisely the kinds of phenomena that are easy to gloss over in ordering-based benchmarks but are central to realistic procedural understanding.

Model ranking is also less stable across tasks than a single average suggests. Several systems improve strongly on Step Reference Resolution and Ingredient Usage Prediction while remaining flat or worse on exact tracing, and some open models trade places with GPT-4o depending on which task is emphasized. That instability is informative rather than a flaw: it shows that the suite separates local dependency recovery from full ingredient-state maintenance instead of collapsing them into one coarse score.

7.2 Failure analysis

Figure 4 shows two typical errors. In the first case, the model predicts that vegetable oil is still present both in the sauce and in the cooked meat, even though the instruction explicitly discards the remaining oil before the sauce is prepared. This is a failure of *subtractive reasoning*: once an ingredient has participated in a step, the model tends to preserve it in all later descendants unless disposal is modeled explicitly.

In the second case, the model identifies soy sauce in the reserved marinade but not in the fried bonito. Here the difficulty comes from *split-and-persist reasoning*. The marinade is first divided into a reserved portion and a portion used to season the fish; the soy sauce therefore persists in both branches. After cooking, the fish should still be traceable to the marinated state. Missing this dependency indicates that models often collapse parallel branches of the ingredient state graph.

7.3 Limitations and recommended use

NHK Recipe State Probing is designed to measure ingredient-centric state tracking, not general cooking competence. It does not evaluate taste, safety, timing control, or grounded action execution. The manually annotated subset is also too small to support definitive ranking claims among closely matched systems. In addition, because the source recipes are public web data, contamination cannot be ruled out for the textual content itself. The benchmark partially mitigates this concern by evaluating latent state annotations that are not present verbatim in the source pages, but it does not constitute a formal decontamination study.

Within those limits, the benchmark is useful in at least three settings: evaluating procedural reasoning in general-purpose LLMs, testing whether multimodal inputs improve state tracking, and studying how direct supervision alters a model's ability to expose latent world-state knowledge [12]. We therefore view the benchmark as a diagnostic tool rather than a leaderboard-only resource.

8 Conclusion

We presented NHK Recipe State Probing, a benchmark for ingredient state tracking in recipes built from professionally edited NHK data and augmented with stepwise world states and transformation links. The benchmark combines large-scale automatic reference resolution with human-annotated usage and tracing tasks, yielding a compact but informative evaluation suite for procedural state understanding. Experiments across text-only and multimodal models show that current systems can exploit surface cues, but still struggle with subtraction, splitting, and persistent ingredient flow. By making these failures measurable, NHK Recipe State Probing provides a practical evaluation framework for future work on grounded procedural reasoning.

AI-Generated

Title: *Early Summer "Ton-ton" Hamburg Steak*
Ingredients:
1/2 cup sake
1/2 teaspoon salt
10g butter
100g ground pork
Vegetable oil
...

Instructions:

Making and pan-frying the hamburger mixture.

⋮

s_8 *Discard the oil remaining in the pan from Step 7, and place the pan over high heat. Add the sake and salt, and reduce the liquid until it thickens slightly.*
Remove the pan from heat, add the butter, and shake the pan to melt the butter and emulsify the sauce.

At the end of Step 8, the ingredients are as follows:
a. Lemon wedges (2/3 of a lemon)
b. Watercress (1 bunch)
c. Sauce
d. Cooked meat, kept warm in aluminum foil

Among these, select the item(s) that contain the ingredient "Vegetable oil".

Gold Standard: d
LLM Response: c, d

Title: *Early Bonito "Arare-yaki"*
Ingredients:
100 ml soy sauce (for marinade)
50 ml sake (for marinade)
...
250 g bonito
Shredded spring cabbage

Instructions:

s_1 *Mix the ingredients for the marinade, and set aside a small amount to use as a dipping sauce. Place the remaining marinade in a plastic bag and add the bonito.*

⋮

s_4 *Remove the bonito from the bag and pat it dry.Coat the entire surface evenly with crushed rice crackers (arare).*

s_5 *Heat 2 to 3 tablespoons of rice oil in a frying pan over high heat.Add the coated bonito (Step 4) and quickly sear both sides until crispy.*

At the end of Step 5, the ingredients are as follows:
a. Remaining marinade
b. Fried bonito
c. Shredded spring cabbage
...

Among these, select the item(s) that contain the ingredient "soy sauce".

Gold Standard: a, b
LLM Response: a

Figure 4: Representative failure cases on Ingredient Tracing. Left: the model incorrectly keeps vegetable oil in the sauce after the recipe explicitly discards the remaining oil from the pan, illustrating difficulty with subtraction and disposal. Right: the model tracks soy sauce to the reserved marinade but misses that it remains in the fried bonito after marination and cooking, illustrating persistence through transformation and split states.

AI-Generated

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).

[2] Weihong Du, Wenrui Liao, Hongru Liang, and Wenqiang Lei. 2024. PAGED: A Benchmark for Procedural Graphs Extraction from Documents. In *ACL (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10829–10846. doi:10.18653/v1/2024.acl-long.583

[3] Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Hiroki Iida, Masanari Ohi, Kakeru Hattori, Hirai Shota, Sakae Mizuki, Rio Yokota, and Naoaki Okazaki. 2024. Continual Pre-Training for Cross-Lingual LLM Adaptation: Enhancing Japanese Language Capabilities. In *COLM*. University of Pennsylvania, USA.

[4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, and Austen Gregerson et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[5] Mahnaz Koupaee and William Yang Wang. 2018. WikiHow: A Large Scale Text Summarization Dataset. arXiv:1810.09305 [cs.CL] https://arxiv.org/abs/1810.09305

[6] Youmi Ma, Sakae Mizuki, Kazuki Fujii, Taishi Nakamura, Masanari Ohi, Hinari Shimada, Taihei Shiotani, Koshiro Saito, Koki Maeda, Kakeru Hattori, Takumi Okamoto, Shigeki Ishida, Rio Yokota, Hiroya Takamura, and Naoaki Okazaki. 2025. Building Instruction-Tuning Datasets from Human-Written Instructions with Open-Weight Large Language Models. arXiv:2503.23714 [cs.CL] https://arxiv.org/abs/2503.23714

[7] Javier Marin, Aritro Biswas, Ferda Ofli, Nicholas Hynes, Amaia Salvador, Yusuf Aytar, Ingmar Weber, and Antonio Torralba. 2019. Recipe1M+: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images. *IEEE Trans. Pattern Anal. Mach. Intell.* (2019).

[8] Liang-Ming Pan, Jingjing Chen, Jianlong Wu, Shaoteng Liu, Chong-Wah Ngo, Min-Yen Kan, Yugang Jiang, and Tat-Seng Chua. 2020. Multi-modal Cooking Workflow Construction for Food Recipes. In *Proceedings of the 28th ACM International Conference on Multimedia* (Seattle, WA, USA) *(MM '20)*. Association for Computing Machinery, New York, NY, USA, 1132–1141. doi:10.1145/3394171.3413765

[9] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning Cross-modal Embeddings for Cooking Recipes and Food Images. In *CVPR*.

[10] Roger C Schank and Robert P Abelson. 2013. *Scripts, plans, goals, and understanding: An inquiry into human knowledge structures*. Psychology press.

[11] Niket Tandon, Keisuke Sakaguchi, Bhavana Dalvi, Dheeraj Rajagopal, Peter Clark, Michal Guerquin, Kyle Richardson, and Eduard Hovy. 2020. A Dataset for Tracking Entities in Open Domain Procedural Text. In *EMNLP*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6408–6417. doi:10.18653/v1/2020.emnlp-main.520

[12] Shubham Toshniwal, Sam Wiseman, Karen Livescu, and Kevin Gimpel. 2022. Baked-in State Probing. In *Findings of EMNLP*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5430–5435. doi:10.18653/v1/2022.findings-emnlp.397

[13] Haoyu Wang, Hongming Zhang, Yueguan Wang, Yuqian Deng, Muhao Chen, and Dan Roth. 2023. Are All Steps Equally Important? Benchmarking Essentiality Detection in Event Processes. In *EMNLP*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4048–4056. doi:10.18653/v1/2023.emnlp-main.246

[14] Te-Lin Wu, Alex Spangher, Pegah Alipoormolabashi, Marjorie Freedman, Ralph Weischedel, and Nanyun Peng. 2022. Understanding Multimodal Procedural Knowledge by Sequencing Multimodal Instructional Manuals. In *ACL (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 4525–4542. doi:10.18653/v1/2022.acl-long.310

[15] Te-Lin Wu, Caiqi Zhang, Qingyuan Hu, Alexander Spangher, and Nanyun Peng. 2023. Learning Action Conditions from Instructional Manuals for Instruction Understanding. In *ACL (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 3023–3043. doi:10.18653/v1/2023.acl-long.170

[16] Semih Yagcioglu, Aykut Erdem, Erkut Erdem, and Nazli Ikizler-Cinbis. 2018. RecipeQA: A Challenge Dataset for Multimodal Comprehension of Cooking Recipes. In *EMNLP*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 1358–1368. doi:10.18653/v1/D18-1166

[17] Yoko Yamakata, Shinsuke Mori, and John Carroll. 2020. English Recipe Flow Graph Corpus. In *LREC*, Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (Eds.). European Language Resources Association, Marseille, France, 5187–5194. https://aclanthology.org/2020.lrec-1.638

[18] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[19] Jeffrey M Zacks and Barbara Tversky. 2001. Event structure in perception and conception. *Psychological bulletin* 127, 1 (2001), 3.

[20] Li Zhang, Qing Lyu, and Chris Callison-Burch. 2020. Reasoning about Goals, Steps, and Temporal Ordering with WikiHow. In *EMNLP*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 4630–4639. doi:10.18653/v1/2020.emnlp-main.374

[21] Shuyan Zhou, Li Zhang, Yue Yang, Qing Lyu, Pengcheng Yin, Chris Callison-Burch, and Graham Neubig. 2022. Show Me More Details: Discovering Hierarchies of Procedures from Semi-structured Web Data. In *ACL (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 2998–3012. doi:10.18653/v1/2022.acl-long.214

[22] Yu Zhou, Sha Li, Manling Li, Xudong Lin, Shih-Fu Chang, Mohit Bansal, and Heng Ji. 2023. Non-Sequential Graph Script Induction via Multimedia Grounding. In *ACL (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 5529–5545. doi:10.18653/v1/2023.acl-long.303

AI-Generated